

B28: SQL SuperHero Program : Session 43

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>

7 May 2025 at

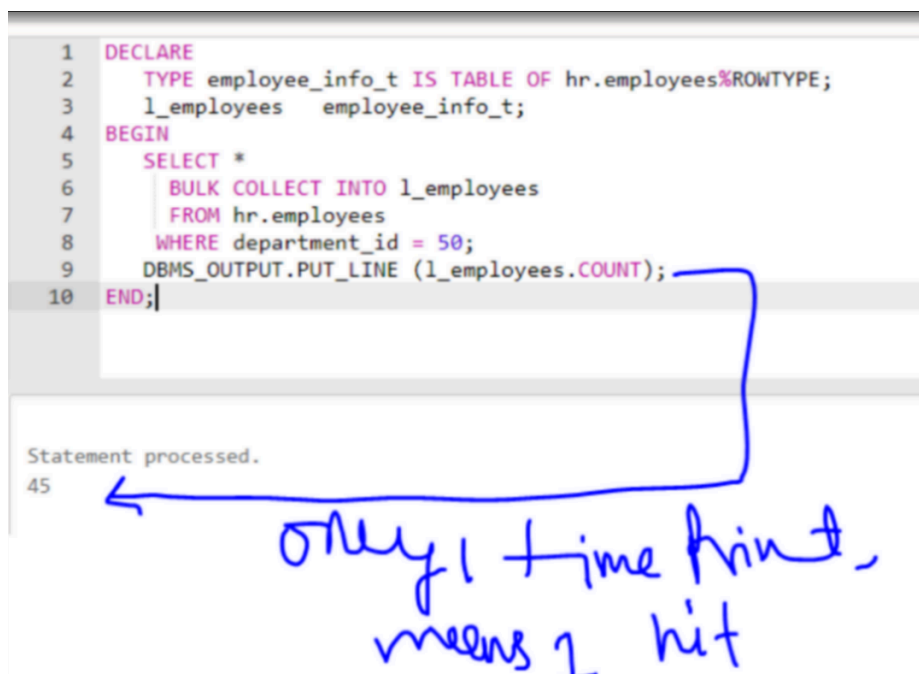
Bcc: satishhattekar1997@gmail.com

BULK COLLECT:

We do not want one by one processing. It is time consuming when we have to process million of records. For performance we have to use the concept of BULK collect, means in 1 shot take all data. I context switch, 1 hit.

BULK COLLECT takes all data in 1 shot and FOR ALL processes all data in 1 shot.

```
DECLARE
TYPE employee_info_t IS TABLE OF hr.employees%ROWTYPE;
l_employees employee_info_t;
BEGIN
SELECT *
  BULK COLLECT INTO l_employees
  FROM hr.employees
 WHERE department_id = 50;
DBMS_OUTPUT.PUT_LINE (l_employees.COUNT);
END;
```



```
1 DECLARE
2   TYPE employee_info_t IS TABLE OF hr.employees%ROWTYPE;
3   l_employees employee_info_t;
4 BEGIN
5   SELECT *
6     BULK COLLECT INTO l_employees
7     FROM hr.employees
8    WHERE department_id = 50;
9   DBMS_OUTPUT.PUT_LINE (l_employees.COUNT);
10  END;
```

Statement processed.

45

only 1 time print,
means 1 hit

--> In the above example, just 1 context switch and take all data in 1 shot and bulk collect into l_employees.

If you do not want to retrieve all the columns in a table, create your own user-defined record type and use that to define your collection. All you have to do is make sure the list of expressions in the SELECT match the record type fields.

```
DECLARE
TYPE two_cols_rt IS RECORD (
  employee_id hr.employees.employee_id%TYPE,
  salary      hr.employees.salary%TYPE
);
TYPE employee_info_t IS TABLE OF two_cols_rt;
l_employees employee_info_t;
BEGIN
SELECT employee_id, salary
  BULK COLLECT INTO l_employees
  FROM hr.employees
 WHERE department_id = 50;
DBMS_OUTPUT.PUT_LINE (l_employees.COUNT);
END;
```

```

1 DECLARE
2     TYPE two_cols_rt IS RECORD (
3         employee_id hr.employees.employee_id%TYPE,
4         salary      hr.employees.salary%TYPE
5     );
6
7     TYPE employee_info_t IS TABLE OF two_cols_rt;
8
9     l_employees employee_info_t;
10 BEGIN
11     SELECT employee_id, salary
12     BULK COLLECT INTO l_employees
13     FROM hr.employees
14     WHERE department_id = 50;
15     DBMS_OUTPUT.PUT_LINE (l_employees.COUNT);
16 END;|

```

*} what can I need
create a Record type*

Statement processed.
45

Let see the same example, I want data from different tables. Like I want employee name, employee salary and department name. I have to bulk process this. Bulk process means, take records & insert in another table.

create table test (name varchar2(50), salary number, depart_name varchar2(50));

```

TYPE insert_data IS RECORD (
    employee_name hr.employees.first_name%TYPE,
    salary        hr.employees.salary%TYPE,
    department_name hr.departments.department_name%type
);

```

--> See the above type, I created. I just created a record that says that yes, I need these 3 columns, which I have to bulk processed.

```

DECLARE
TYPE insert_data IS RECORD (
    employee_name hr.employees.first_name%TYPE,
    salary        hr.employees.salary%TYPE,
    department_name hr.departments.department_name%type
);

TYPE insert_data_t IS TABLE OF insert_data;

insert_data_t_aa insert_data_t;
BEGIN
SELECT E.first_name, E.salary, D.department_name
BULK COLLECT INTO insert_data_t_aa
FROM hr.employees E, hr.departments D
WHERE D.department_id = E.department_id;

forall x in insert_data_t_aa.first .. insert_data_t_aa.last
Insert into test values insert_data_t_aa(x);
END;

```

```

1 DECLARE
2     TYPE insert_data IS RECORD (
3         employee_name hr.employees.first_name%TYPE,
4         salary         hr.employees.salary%TYPE,
5         department_name hr.departments.department_name%type
6     );
7
8     TYPE insert_data_t IS TABLE OF insert_data;
9
10    insert_data_t_aa insert_data_t;
11 BEGIN
12     SELECT E.first_name, E.salary, D.department_name
13     BULK COLLECT INTO insert_data_t_aa
14     FROM hr.employees E, hr.departments D
15     WHERE D.department_id = E.department_id;
16
17     forall x in insert_data_t_aa.first .. insert_data_t_aa.last
18     Insert into test values insert_data_t_aa(x);
19 END;
20
21

```

Handwritten annotations:

- Line 2-5: **Record** (indicated by a bracket and arrow)
- Line 8: **Type of record** (indicated by an arrow)
- Line 10: **object** (indicated by an arrow)
- Line 13-15: **All data Bulk collect in 1 shot** (indicated by a bracket and arrow)
- Line 17-18: **All inserted in 1 shot** (indicated by a bracket and arrow)

Difference between forall and for loop:

- For LOOP statement executes a sequence of statements multiple times. You would use a FOR LOOP when you want to execute the loop body a fixed number of times.
- The FORALL statement issues a series of static or dynamic DML statements, usually much faster than an equivalent FOR loop, requires some setup code, because each iteration of the loop must use values from one or more collections in its VALUES or WHERE clauses
- The FORALL allows us to bind the contents of a collection to a single DML statement, allowing the DML to be run for each row in the collection without requiring a context switch each time

LIMIT Clause In Bulk Collection

Whenever we retrieve or fetch a large number of records using bulk collect clause, our program starts consuming a lot of memory in order to be fast and efficient. This degrades the performance of the database. This means that our query must surely be performing well but at the same time, our database may not. Using an explicit cursor and the LIMIT clause of the BULK COLLECT syntax to limit the total number of rows returned.

As LIMIT works as an attribute of the FETCH-INTO statement thus to use it you can add keyword LIMIT followed by a specific numerical digit which will specify the number of rows that the bulk-collect clause will retrieve in one go at the end of FETCH-INTO statement.

FETCH <cursor_name> BULK COLLECT INTO <plsql_collection> LIMIT number;

As the number of rows in the table grows, more memory will be consumed. So the general recommendation for production code, working with tables that may grow greatly in size, is to avoid SELECT BULK COLLECT INTO (an implicit query) and instead use the FETCH BULK COLLECT with a LIMIT clause.

we cannot use the LIMIT clause with SELECT-INTO statement. LIMIT clause works as an attribute to the FETCH-INTO statement because LIMIT clause requires an explicit cursor to work and FETCH-INTO statement is a part of explicit cursor life cycle. So always remember the LIMIT clause can only be used when you are using BULK COLLECT with FETCH-INTO statement. It cannot be used when you are using bulk collect with SELECT-INTO statement.

Without LIMIT:

```

DECLARE
    TYPE employees_t IS TABLE OF hr.employees%ROWTYPE;
    l_employees employees_t;
BEGIN
    SELECT *
    BULK COLLECT INTO l_employees
    FROM hr.employees;

    DBMS_OUTPUT.put_line (l_employees.COUNT);

    FOR indx IN 1 .. l_employees.COUNT
    LOOP
        DBMS_OUTPUT.put_line (l_employees (indx).last_name);
    END LOOP;
END;

```

```

1 DECLARE
2     TYPE employees_t IS TABLE OF hr.employees%ROWTYPE;
3     l_employees employees_t;
4 BEGIN
5     SELECT *
6     BULK COLLECT INTO l_employees
7     FROM hr.employees;
8
9     DBMS_OUTPUT.put_line (l_employees.COUNT);
10
11    FOR indx IN 1 .. l_employees.COUNT
12    LOOP
13        DBMS_OUTPUT.put_line (l_employees (indx).last_name);
14    END LOOP;
15 END;

```

Statement processed.
107
King
Kochhar
De Haan

All records in 1 shot)

→ Total records

```

DECLARE

CURSOR emp_cur IS SELECT * FROM hr.employees;

TYPE employee_aat IS TABLE OF hr.employees%ROWTYPE;
l_employee employee_aat;
BEGIN
OPEN emp_cur;

LOOP
FETCH emp_cur BULK COLLECT INTO l_employee LIMIT 20;
EXIT WHEN l_employee.COUNT = 0;

DBMS_OUTPUT.put_line ('Fetched ' || l_employee.COUNT);

FOR indx IN 1 .. l_employee.COUNT
LOOP
DBMS_OUTPUT.put_line (
l_employee (indx).employee_id || ' and name is ' ||
l_employee (indx).last_name);
END LOOP;
END LOOP;

CLOSE emp_cur;
END;

```

```

DECLARE
    cursor emp_cur is
    select * from hr.employees;

    type employee_info_t is table of hr.employees%rowtype;
    l_employees employee_info_t;

BEGIN
    open emp_cur;
    Loop
        Fetch emp_cur Bulk Collect into l_employees Limit 20;
        exit when l_employees.count = 0;

        dbms_output.put_line('Total Fetched ' || l_employees.count);

        For I in 1 .. l_employees.count Loop
            dbms_output.put_line( l_employees(i).employee_id || ' and name is ' || l_employees(i).first_name);
        End Loop;
    End Loop;

END;

```

20 every 5 hot.
1 context switch
= 20 records

Exit
when
nothing
left

```

Statement processed.
Fetched 20
100 and name is King
101 and name is Kochhar
102 and name is De Haan
103 and name is Hunold
104 and name is Ernst
105 and name is Austin
106 and name is Pataballa
107 and name is Lorentz
108 and name is Greenberg
109 and name is Faviet
110 and name is Chen
111 and name is Sciarra
112 and name is Urman
113 and name is Popp
114 and name is Raphaely
115 and name is Khoo
116 and name is Baida
117 and name is Tobias
118 and name is Himuro
119 and name is Colmenares
Fetched 20
120 and name is Weiss
121 and name is Fripp
122 and name is Kaufling
123 and name is Vollman
124 and name is Mourgos

```

20 fetch in one shot

```

=====
DECLARE
  cursor emp_cur is
  select * from hr.employees;

  type employee_info_t is table of hr.employees%rowtype;
  l_employees employee_info_t;
  cnt number := 0;
BEGIN
  open emp_cur;
  Loop
    Fetch emp_cur Bulk Collect into l_employees Limit 20;
    exit when emp_cur%notfound;

    dbms_output.put_line('Total Fetched '||l_employees.count);

    For I in 1 .. l_employees.count Loop
      cnt := cnt+1;
      dbms_output.put_line(cnt||' - '||l_employees(i).employee_id||' and name is '||l_employees(i).first_name);
    End Loop;
  End Loop;
END;

```

```

DECLARE
  cursor emp_cur is
  select * from hr.employees;

  type employee_info_t is table of hr.employees%rowtype;
  l_employees employee_info_t;
  cnt number := 0;
BEGIN
  open emp_cur;
  Loop
    Fetch emp_cur Bulk Collect into l_employees Limit 20;
    exit when emp_cur%notfound;

    dbms_output.put_line('Total Fetched '||l_employees.count);

    For I in 1 .. l_employees.count Loop
      cnt := cnt+1;
      dbms_output.put_line(cnt||' - '||l_employees(i).employee_id||' and name is '||l_employees(i).first_name);
    End Loop;
  End Loop;
END;

```

go in one shot

Exit when false

```

92 - 191 and name is Randall
93 - 192 and name is Sarah
94 - 193 and name is Britney
95 - 194 and name is Samuel
96 - 195 and name is Vance
97 - 196 and name is Alana
98 - 197 and name is Kevin
99 - 198 and name is Donald
100 - 199 and name is Douglas

```

only 100
Records
Printed

```
select Count(*) from hr.employees
```

COUNT(*)

107

→ We have a total
107 records

Now see this above example. The main concern on Line No. 9 and 10.

```

fetch c1 bulk collect into t limit 20;
exit when c1%notfound;

```

Total 107 records, fetched in 20 per shot. and next line says exit when not found. now see the above output. It only displayed 100 records, where are my last 7 rows? The problem is we are using "exit when c1%notfound;" just after bulk collect. In the 6th iteration($20 \times 5 + 7$) when Oracle bulk collected the last 7 rows and cursor got exhausted, Oracle Set c1%notfound as true and we exited from our for loop.

so this is wrong. How to fix it?

```

DECLARE
    cursor emp_cur is
        select * from hr.employees;

    type employee_info_t is table of hr.employees%rowtype;
    l_employees employee_info_t;
    cnt number := 0;
BEGIN
    open emp_cur;
    loop
        Fetch emp_cur Bulk Collect into l_employees Limit 8;
        exit when emp_cur%notfound;
        dbms_output.put_line('Total Fetched ' || l_employees.count);

        For I in 1 .. l_employees.count Loop
            cnt := cnt+1;
            dbms_output.put_line(cnt || ' - ' || l_employees(i).employee_id || ' and name is ' || l_employees(i).first_name);
        End Loop;
    End Loop;
END;

```

→ 8 in one shot.

→ exit when true

```

Total Fetched 8
97 - 196 and name is Alana
98 - 197 and name is Kevin
99 - 198 and name is Donald
100 - 199 and name is Douglas
101 - 200 and name is Jennifer
102 - 201 and name is Michael
103 - 202 and name is Pat
104 - 203 and name is Susan

```

✓ 104 records, 3 is missing

Same concept. Total 107 records. every shot 8 records. so 13 iteration of 8, $13 \times 8 = 104$ and next 3 is left, because %NotFound is set to true.

Solution:

```

DECLARE
  cursor emp_cur is
  select * from hr.employees;

  type employee_info_t is table of hr.employees%rowtype;
  l_employees employee_info_t;
  cnt number := 0;
BEGIN
  open emp_cur;
  Loop
    Fetch emp_cur Bulk Collect into l_employees Limit 8;
    exit when l_employees.count = 0;
    dbms_output.put_line('Total Fetched ' || l_employees.count);

    For I in 1 .. l_employees.count Loop
      cnt := cnt+1;
      dbms_output.put_line(cnt || ' - ' || l_employees(i).employee_id || ' and name is ' || l_employees(i).first_name);
    End Loop;
  End Loop;
END;

```

use this

How to Pick the Limit for BULK COLLECT

When you use BULK COLLECT, you retrieve more than row with each fetch, reducing context switching between the SQL and PL/SQL engines, and thereby improving performance. That collection consumes per-session Process Global Area memory or PGA. So you face a typical tradeoff when it comes to performance optimization: reduce CPU cycles, but use more memory. Start with 100 as limit and see if that's fast enough. If not, try higher values until you reach the performance level you need - and you are not consuming too much PGA memory.

There is no such formula to determine LIMIT value. Start with a value like 500 or 1000 and see the performance impact and it is hit and trial.